Université Hassan II de Casablanca

Faculté des Sciences Ben M'Sik (FSBM)

Filière : Génie Informatique — Semestre 6

Module : Développement Full Stack

RAPPORT DE PROJET

Conception et Réalisation d'une Plateforme E-Commerce Full Stack

« E-Store »

Spring Boot 3 · JPA · MongoDB · Angular 18

Réalisé par :
- **Adnane Louardi**
- **Yassine Boukhary**

Encadré par :
- **Pr. Omar Zahour**

Année universitaire : 2025–2026

Dédicace

À nos familles, pour leur soutien indéfectible tout au long de notre parcours académique. À nos amis et camarades de promotion, qui ont rendu cette aventure plus riche et plus joyeuse. À Pr. Omar Zahour, dont les conseils avisés et la rigueur pédagogique ont guidé chaque étape de ce projet. Ce travail vous est dédié.

Remerciements

Nous tenons à exprimer notre profonde gratitude à Pr. Omar Zahour pour avoir proposé ce projet stimulant et pour ses précieuses orientations techniques tout au long du semestre. Nos remerciements vont également à l'ensemble du corps enseignant de la Faculté des Sciences Ben M'Sik pour la qualité de la formation dispensée. Enfin, nous remercions chaleureusement nos familles pour leur patience et leur encouragement constants durant cette période de travail intensif.

Résumé

Ce rapport présente la conception et la réalisation d'une plateforme e-commerce complète intitulée E-Store, développée dans le cadre du module Full Stack du semestre 6 à la Faculté des Sciences Ben M'Sik. L'application adopte une architecture en couches découplée : un backend REST construit avec Spring Boot 3.3.5 et Java 21, organisé en six domaines métier (Customer, Catalog, Inventory, Shopping, Billing, Review), et un frontend Angular 18 composé de composants standalone. La persistance est hybride : Spring Data JPA avec H2 (développement) ou MySQL (production) pour les données relationnelles, et MongoDB pour les avis produit. La sécurité repose sur JSON Web Tokens (JWT) et le chiffrement BCrypt. Le projet est documenté via Swagger/OpenAPI et validé par un script de smoke-test couvrant l'intégralité des 28 endpoints implémentés.

Abstract

This report describes the design and implementation of a full e-commerce platform called E-Store, developed as part of the Full Stack module (semester 6) at the Faculty of Sciences Ben M'Sik. The application follows a layered, domain-driven architecture: a REST backend built with Spring Boot 3.3.5 and Java 21, structured into six business domains (Customer, Catalog, Inventory, Shopping, Billing, Review), and an Angular 18 frontend made of standalone components. Persistence is hybrid: Spring Data JPA with H2 (development) or MySQL (production) for relational data, and MongoDB for product reviews. Security is based on JWT and BCrypt hashing. The project is documented via Swagger/OpenAPI and validated by a smoke-test script covering all 28 implemented endpoints.

ملخص

يصف هذا التقرير تصميم وتنفيذ منصة تجارة إلكترونية متكاملة تُسمى "المتجر الإلكتروني"، طُوّرت كجزء من وحدة "التطوير الشامل" (الفصل الدراسي السادس) في كلية العلوم بن مسيك. يتبع التطبيق بنيةً متعددة الطبقات وموجهة حسب المجال: واجهة خلفية REST مبنية باستخدام Spring Boot 3.3.5 وJava 21، مُقسّمة إلى ستة مجالات أعمال (العملاء، الكتالوج، المخزون، التسوق، الفواتير، المراجعات)، وواجهة أمامية Angular 18 تتكون من مكونات مستقلة. تعتمد آلية التخزين على نظام هجين : Spring Data JPAمع H2 (للتطوير) أو MySQL (للإنتاج) للبيانات العلائقية، و MongoDBلمراجعات المنتجات. يعتمد الأمان على JWTوتجزئة BCrypt. تم توثيق المشروع باستخدامSwagger/OpenAPI ، وتم التحقق من صحته بواسطة برنامج اختبار سريع يغطي جميع نقاط النهاية الـ 28 المُنفذة.

Table des matières

Cliquez-droit sur le champ ci-dessous puis « Mettre à jour les champs » pour générer la table.

Liste des figures

- **Figure 3.1 —** Architecture en couches de E-Store
- **Figure 3.2 —** Décomposition par domaines métier
- **Figure 3.3 —** Architecture technique (Angular ↔ Spring Boot ↔ BDD)
- **Figure 3.4 —** Diagramme de cas d'utilisation global
- **Figure 3.5 —** Diagramme de classes (JPA + MongoDB)
- **Figure 3.6 —** Diagramme Entité-Relation (9 tables)
- **Figure 3.7 —** Diagramme de séquence — Authentification JWT
- **Figure 3.8 —** Diagramme de séquence — Checkout transactionnel
- **Figure 3.9 —** Diagramme de séquence — Ajout panier (vérification stock)
- **Figure 3.10 —** Diagramme de séquence — Dépôt d'avis (MongoDB)
- **Figure 6.1 —** Page d'accueil — liste des produits
- **Figure 6.2 —** Détail d'un produit et avis
- **Figure 6.3 —** Formulaire d'inscription
- **Figure 6.4 —** Formulaire de connexion
- **Figure 6.5 —** Panier d'achat
- **Figure 6.6 —** Page de confirmation de commande
- **Figure 6.7 —** Historique des commandes
- **Figure 6.8 —** Profil utilisateur
- **Figure 6.9 —** Dashboard administrateur
- **Figure 6.10 —** Gestion des stocks (admin)

Liste des tableaux

- **Tableau 2.1 —** Acteurs et leurs droits
- **Tableau 2.2 —** Exigences fonctionnelles par domaine
- **Tableau 2.3 —** Exigences non fonctionnelles
- **Tableau 3.1 —** Entités JPA et relations
- **Tableau 4.1 —** Dépendances Maven du projet
- **Tableau 5.1 —** Structure du backend (76 classes / 9 packages)
- **Tableau 5.2 —** Structure du frontend (17 composants / 8 services)
- **Tableau 7.1 —** Tests fonctionnels par domaine
- **Tableau 7.2 —** Tests de sécurité
- **Tableau A.1 —** Inventaire complet des 28 endpoints REST

Introduction Générale

Le commerce électronique est aujourd'hui l'un des secteurs numériques les plus dynamiques, et la maîtrise de sa réalisation technique constitue une compétence clé pour tout ingénieur logiciel. Dans ce contexte, le projet E-Store propose de concevoir et de réaliser une plateforme e-commerce complète et fonctionnelle, couvrant la totalité de la chaîne : catalogue produit, gestion des stocks, panier, commande transactionnelle, et avis clients.

Problématique

Comment architecturer une application web full stack de manière à séparer clairement les responsabilités (frontend / backend / persistance), à gérer plusieurs bases de données de nature différente (relationnelle et document), et à assurer la sécurité des accès tout en restant maintenable et portable entre environnements ?

Objectifs du projet

- Mettre en œuvre une API REST complète avec Spring Boot 3 et Java 21.
- Organiser le backend en domaines métier cohérents (architecture par domaines).
- Intégrer une persistance hybride : JPA/H2/MySQL pour les données relationnelles, MongoDB pour les avis.
- Sécuriser l'ensemble des endpoints avec JWT et BCrypt.
- Développer un frontend Angular 18 découplé, avec composants standalone et lazy loading.
- Documenter et tester l'API via Swagger/OpenAPI et un script de smoke-test.

Organisation du rapport

Ce document est structuré en sept chapitres. Le Chapitre 1 présente le cadre général du projet. Le Chapitre 2 couvre l'analyse et la spécification des besoins. Le Chapitre 3 détaille la conception UML. Le Chapitre 4 décrit les technologies choisies. Le Chapitre 5 présente l'implémentation concrète. Le Chapitre 6 illustre les interfaces utilisateur. Le Chapitre 7 rapporte les tests et la validation. Une conclusion et des annexes complètent le document.

Chapitre 1 — Cadre du projet

1.1 Contexte académique

Ce projet s'inscrit dans le cadre du module « Développement Full Stack » du semestre 6 de la filière Génie Informatique à la Faculté des Sciences Ben M'Sik (FSBM), Université Hassan II de Casablanca. Il est supervisé par Pr. Omar Zahour et constitue le projet de fin de module évalué sur 20 points selon le barème du cahier des charges.

1.2 Sujet : E-Store

Le sujet imposé est la réalisation d'un mini-projet e-commerce « E-Store » intégrant : un backend Spring Boot avec Spring Data JPA pour la persistance relationnelle, Spring Data MongoDB pour les avis, Spring Security pour l'authentification JWT, et un frontend Angular avec gestion de l'état et des formulaires réactifs.

1.3 Objectifs pédagogiques

- Maîtriser le développement d'une API REST avec Spring Boot 3 et Java 21.
- Comprendre la persistance hybride (JPA + MongoDB).
- Implémenter un mécanisme d'authentification stateless (JWT).
- Développer un frontend Angular 18 moderne avec composants standalone.
- Pratiquer la méthodologie incrémentale (développement jour par jour).

1.4 Méthodologie adoptée

Le développement a suivi une approche incrémentale structurée en six itérations journalières : Jour 1 (infrastructure + entités JPA), Jour 2 (authentification JWT), Jour 3 (catalogue), Jour 4 (stocks), Jour 5 (panier + commandes), Jour 6 (avis MongoDB + frontend Angular). Chaque itération a produit des fonctionnalités testables, permettant une validation continue par le script de smoke-test.

1.5 Barème d'évaluation

Critère	Points	Description
Architecture & conception	4	Couches, domaines, UML, ERD
Backend JPA (Spring Boot)	4	Entités, repos, services, sécurité JWT
Frontend Angular	3	Composants, routing, guards, intercepteurs
Intégration MongoDB	2	Reviews document, fallback startup
Qualité du rapport	2	Structure, contenu, diagrammes, bibliographie
Tests & validation	5	Smoke-test, Swagger, cas de test

Chapitre 2 — Analyse et Spécification des Besoins

2.1 Acteurs du système

Acteur	Rôle	Droits
Visiteur	Utilisateur non authentifié	Parcourir catalogue, rechercher, s'inscrire, se connecter
Utilisateur (USER)	Client authentifié	Panier, commandes, profil, avis
Administrateur (ADMIN)	Gestionnaire du site	CRUD catégories/produits, stocks, toutes commandes

2.2 Besoins fonctionnels

2.2.1 Customer & Authentification

- Inscription avec email + mot de passe (hashé BCrypt).
- Connexion avec retour d'un token JWT valable 24 heures.
- Consultation et mise à jour du profil (téléphone, adresse, ville, pays).
- Liste des utilisateurs (ADMIN uniquement).

2.2.2 Catalogue

- CRUD complet des catégories (ADMIN).
- CRUD complet des produits avec image URL et prix (ADMIN).
- Recherche paginée avec filtres : mot-clé, catégorie, tri, pagination (PUBLIC).
- Consultation du détail d'un produit incluant le stock disponible (PUBLIC).

2.2.3 Stocks (Inventory)

- Chaque produit dispose d'un enregistrement d'inventaire automatiquement créé à 0.
- Consultation publique du stock d'un produit.
- Mise à jour du stock par l'ADMIN.

2.2.4 Panier (Shopping)

- Panier créé automatiquement au premier accès d'un utilisateur authentifié.
- Ajout d'un article avec vérification du stock disponible (422 si dépassement).
- Modification de la quantité d'un article du panier.
- Suppression d'un article ou vidage complet du panier.

2.2.5 Commandes (Billing)

- Passage de commande atomique (@Transactional) : création, décrémentation stock, vidage panier.
- Consultation de ses commandes (utilisateur) ou de toutes (ADMIN), paginée.
- Modification d'une ligne de commande (avant expédition).
- Annulation d'une commande (statut → CANCELLED).

2.2.6 Avis (Reviews — MongoDB)

- Lecture publique des avis d'un produit.
- Dépôt d'un avis authentifié (note 1–5 + commentaire).

- Démarrage gracieux si MongoDB est indisponible.

2.3 Besoins non fonctionnels

Exigence	Détail
Sécurité	JWT stateless (exp. 24h), BCrypt strength 10, CORS restreint à localhost:4200
Performance	Pagination Spring Data (20/page défaut), indexes BDD (idx_products_name, idx_products_category)
Maintenabilité	Architecture en couches + domaines, MapStruct pour mapping, Lombok pour réduction boilerplate
Portabilité	Profils Spring : dev (H2 fichier) / mysql (MySQL 8), configuration externalisée via application.yml
Testabilité	Swagger UI /swagger-ui.html, script smoke-test.sh (28 assertions), collection Postman
Documentation API	Springdoc-openapi 2.6.0, schéma Bearer JWT configuré dans OpenApiConfig

2.4 Cas d'utilisation détaillés

UC01 — S'inscrire

Cas d'utilisation	UC01 — S'inscrire
Acteur(s)	Visiteur
Précondition	L'email n'est pas encore enregistré
Flux principal	1. Le visiteur saisit prénom, nom, email, mot de passe. 2. Le système valide les champs (@Valid). 3. BCrypt encode le mot de passe. 4. Le système crée l'utilisateur (rôle USER) et un profil vide. 5. Un token JWT est retourné.
Exception / flux alternatif	Email déjà utilisé → 422 Unprocessable Entity

UC02 — Ajouter un produit au panier

Cas d'utilisation	UC02 — Ajouter un produit au panier
Acteur(s)	Utilisateur authentifié
Précondition	L'utilisateur est connecté (token JWT valide), le produit existe
Flux principal	1. L'utilisateur sélectionne un produit et une quantité. 2. Le système vérifie le stock disponible. 3. Si OK : l'article est ajouté (ou mis à jour si déjà présent), avec snapshot du prix. 4. Le CartResponse est renvoyé avec le total recalculé.
Exception / flux alternatif	Quantité > stock → 422 BusinessException

UC03 — Passer une commande (Checkout)

Cas d'utilisation	UC03 — Passer une commande (Checkout)
Acteur(s)	Utilisateur authentifié
Précondition	Le panier est non vide, stock suffisant pour chaque article

Flux principal	1. L'utilisateur confirme la commande. 2. @Transactional : Order créée, OrderItems snapshottés, stocks décrémentés, panier vidé. 3. Statut CONFIRMED, OrderResponse retourné.
Exception / flux alternatif	Panier vide → 422; Stock insuffisant → 422 (rollback total)

UC04 — Gérer le catalogue (ADMIN)

Cas d'utilisation	UC04 — Gérer le catalogue (ADMIN)
Acteur(s)	Administrateur
Précondition	Token JWT avec rôle ADMIN
Flux principal	1. L'admin crée/modifie/supprime une catégorie ou un produit. 2. Validation @Valid des champs. 3. MapStruct convertit DTO ↔ entité. 4. Persistance JPA, réponse DTO.
Exception / flux alternatif	Catégorie en double → 422; Produit introuvable → 404

Chapitre 3 — Conception

3.1 Introduction à la modélisation UML

La conception de E-Store s'appuie sur le langage UML 2 pour décrire l'architecture, les entités et les interactions du système. Les diagrammes présentés dans ce chapitre ont été générés avec Mermaid et constituent le référentiel de conception du projet. Ils ont été validés contre le code source pour garantir leur cohérence.

3.2 Architecture logicielle

3.2.1 Architecture en couches

L'application adopte une architecture classique à trois couches, conforme aux recommandations Spring Boot : la couche Présentation (Angular 18), la couche Métier (Spring Boot), et la couche Données (JPA + MongoDB). Cette séparation garantit l'indépendance des couches et facilite la maintenance.

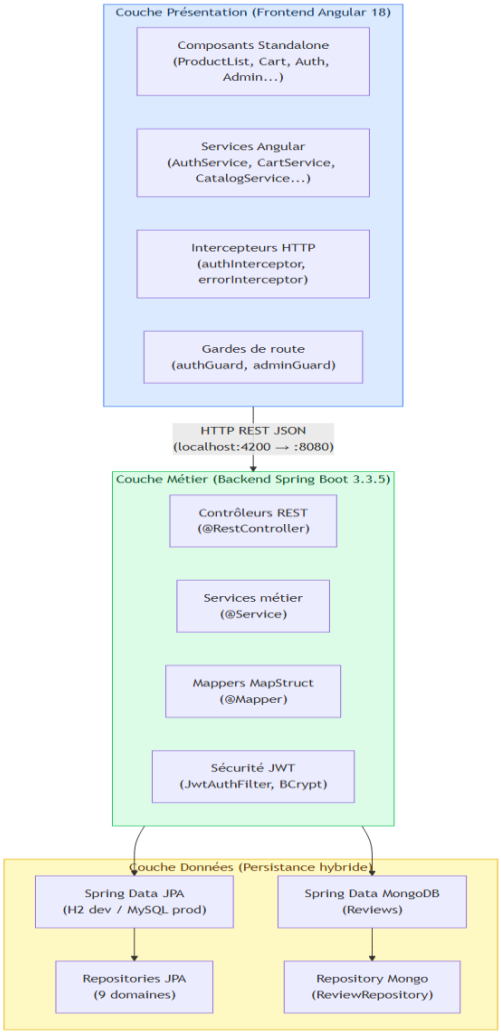


Figure 3.1 — Architecture en couches de E-Store

3.2.2 Architecture par domaines

Le backend est subdivisé en six domaines métier indépendants, chacun encapsulant ses propres entités, repositories, services, mappers, DTOs et contrôleurs. Cette approche Domain-Driven réduit le couplage inter-modules et facilite l'évolution indépendante de chaque domaine.

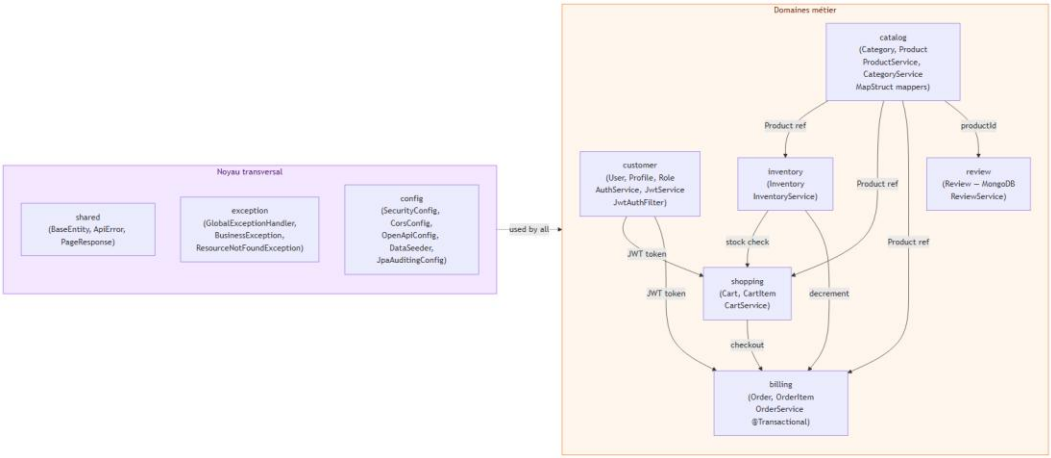


Figure 3.2 — Décomposition par domaines métier

3.3 Diagramme de cas d'utilisation

Le diagramme ci-dessous recense l'ensemble des fonctionnalités offertes par le système, regroupées par acteur. On distingue les fonctionnalités publiques accessibles sans authentification, les fonctionnalités réservées aux utilisateurs connectés, et les fonctionnalités d'administration.

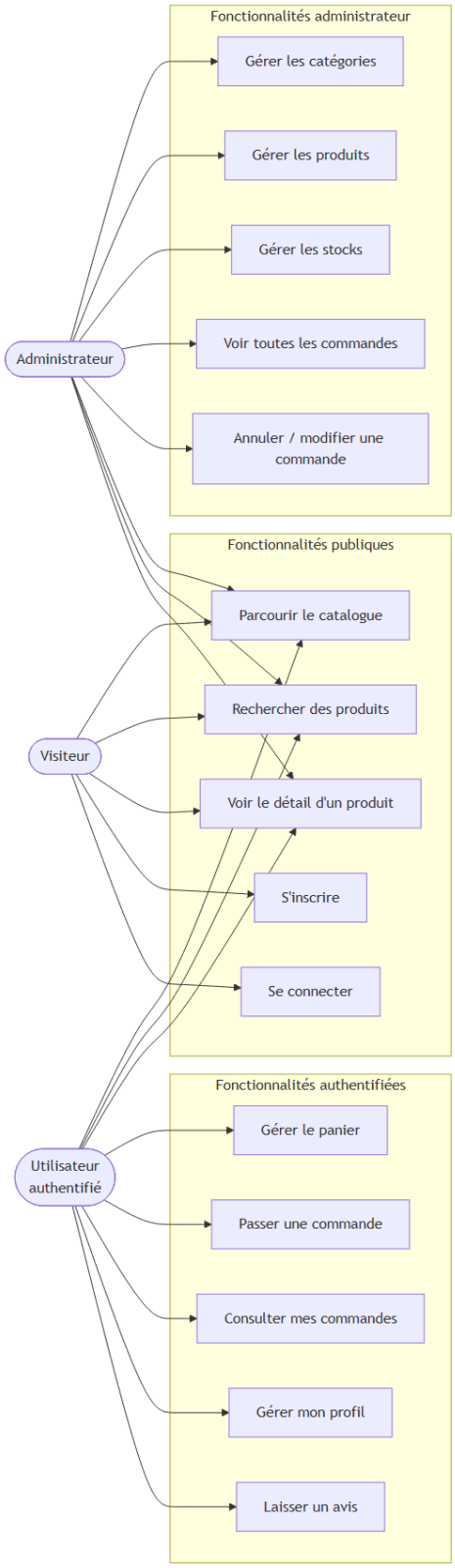


Figure 3.4 — Diagramme de cas d'utilisation global

3.4 Diagramme de classes

Le diagramme de classes présente l'ensemble des entités du projet et leurs relations. Les entités JPA héritent toutes de BaseEntity (id, createdAt, updatedAt via JPA Auditing). L'entité Review est un document MongoDB, sans héritage de BaseEntity, identifié par un String id généré par MongoDB.

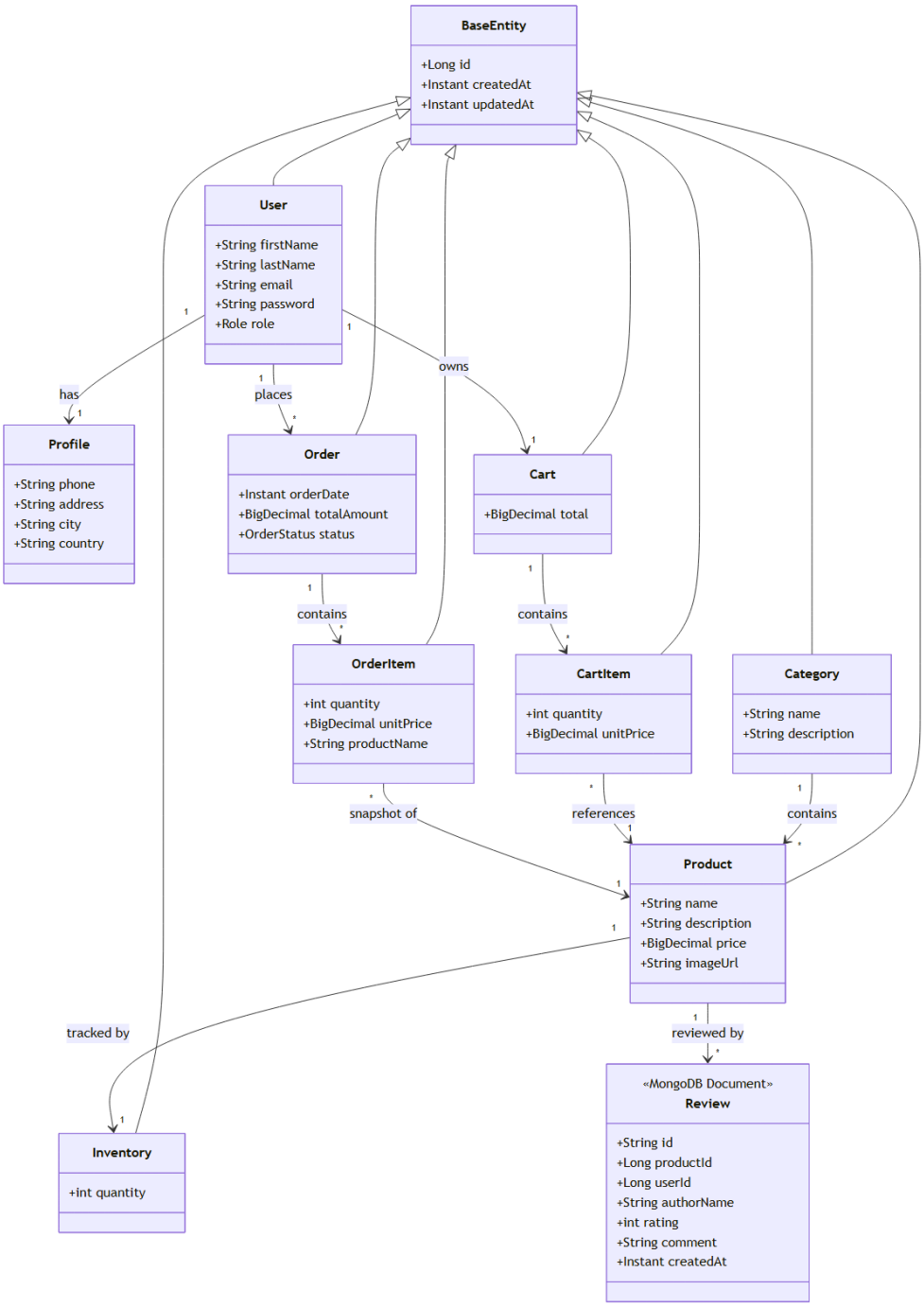


Figure 3.5 — Diagramme de classes (JPA + MongoDB)

3.4.1 Relations JPA

Entité source	Entité cible	Type	Détail
User	Profile	@OneToOne (cascade ALL)	Profil créé à l'inscription
User	Cart	@OneToOne (cascade ALL)	Panier créé automatiquement
User	Order	@OneToMany	Historique des commandes
Category	Product	@OneToMany	Produits d'une catégorie
Product	Inventory	@OneToOne (cascade ALL)	Stock créé à la création produit
Cart	CartItem	@OneToMany (cascade ALL)	Articles du panier
CartItem	Product	@ManyToOne	Référence produit avec snapshot prix
Order	OrderItem	@OneToMany (cascade ALL)	Lignes de commande

3.5 Diagramme Entité-Relation (ERD)

Le modèle relationnel comprend 9 tables. Les clés primaires sont de type BIGINT auto-incrémenté. Les colonnes email (users) et name (categories) disposent de contraintes UNIQUE. Les tables cart_items et order_items stockent le prix unitaire au moment de l'opération (snapshot), découplant ainsi le catalogue des transactions passées.

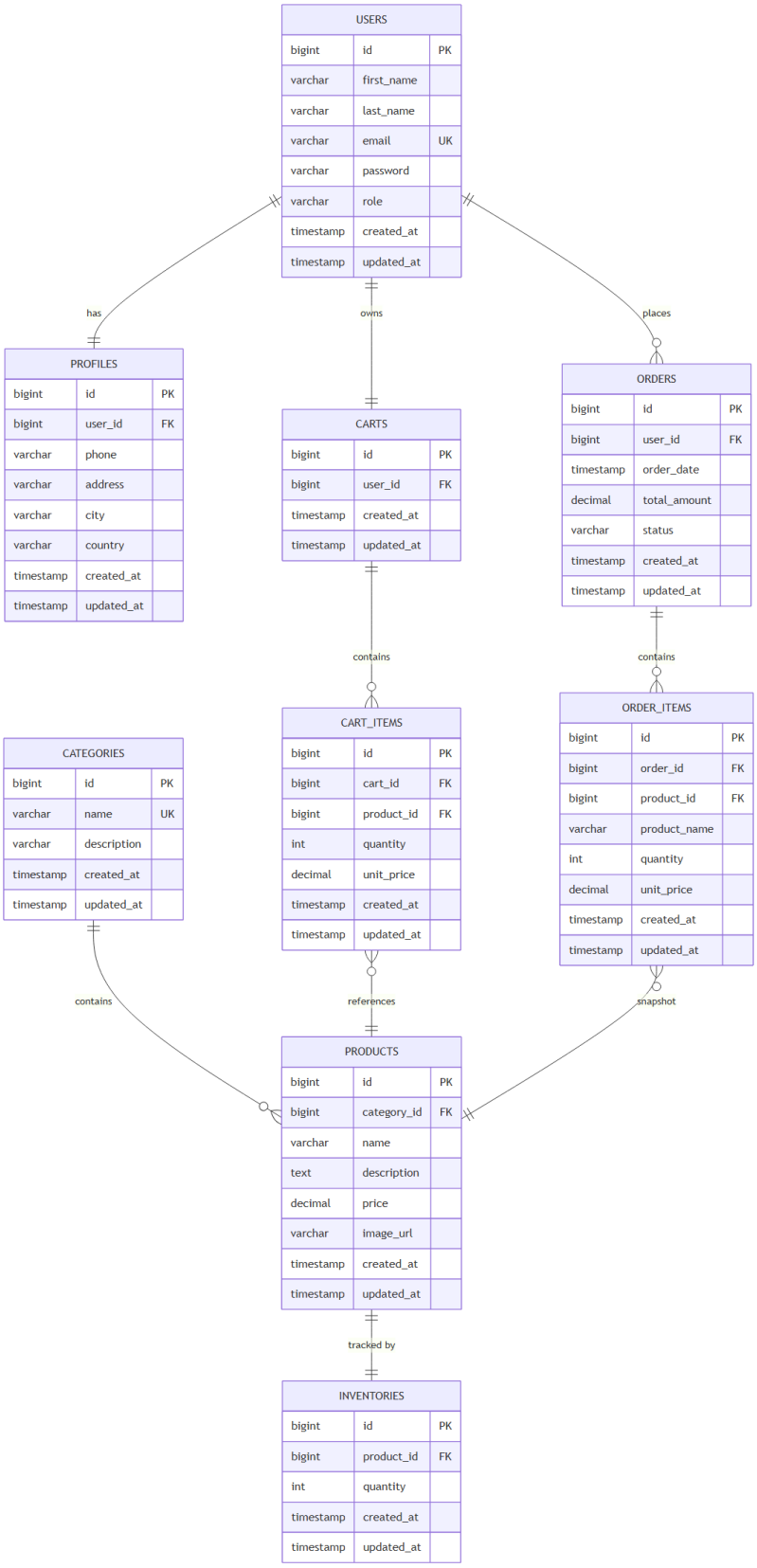


Figure 3.6 — Diagramme Entité-Relation (9 tables)

3.6 Justification de la persistance hybride (JPA vs MongoDB)

Spring Data JPA est retenu pour les données relationnelles car elles impliquent de nombreuses jointures (User ↔ Cart ↔ CartItem ↔ Product, Order ↔ OrderItem) et des transactions ACID. MongoDB est choisi pour les avis produit car leur structure est document-centrique (pas de jointures nécessaires), leur volume peut croître indépendamment des autres entités, et leur indisponibilité ne doit pas bloquer le démarrage de l'application. Ce choix illustre l'approche polyglotte préconisée par le cahier des charges.

3.7 Diagrammes de séquence

3.7.1 DS1 — Authentification JWT

Ce diagramme illustre le flux d'authentification. Après vérification des credentials via BCrypt, le JwtService génère un token signé HS256, valable 24 heures. Le frontend stocke ce token dans le localStorage et l'injecte dans chaque requête protégée via l'authInterceptor Angular.

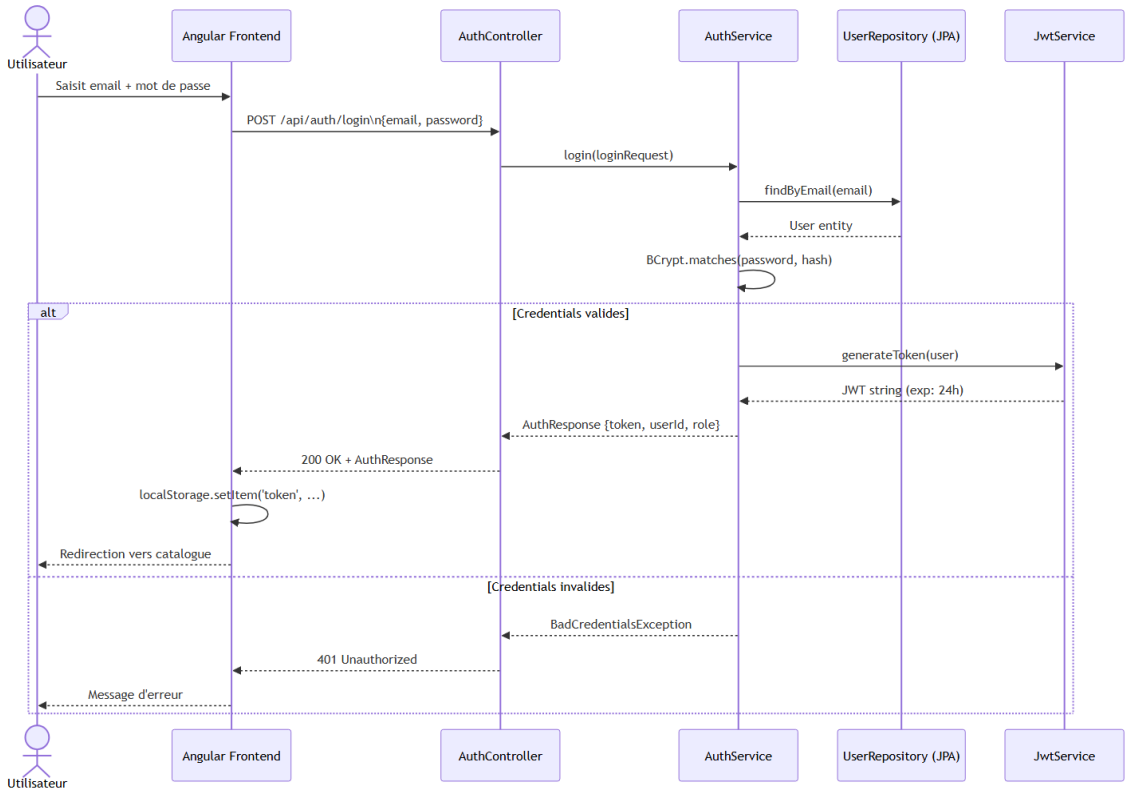


Figure 3.7 — Diagramme de séquence — Authentification JWT

3.7.2 DS3 — Ajout au panier (vérification stock)

L'ajout d'un article au panier implique une vérification systématique du stock disponible avant toute modification. Si la quantité demandée dépasse le stock, une BusinessException est levée et le panier n'est pas modifié. Le prix unitaire est snapshotté au moment de l'ajout.

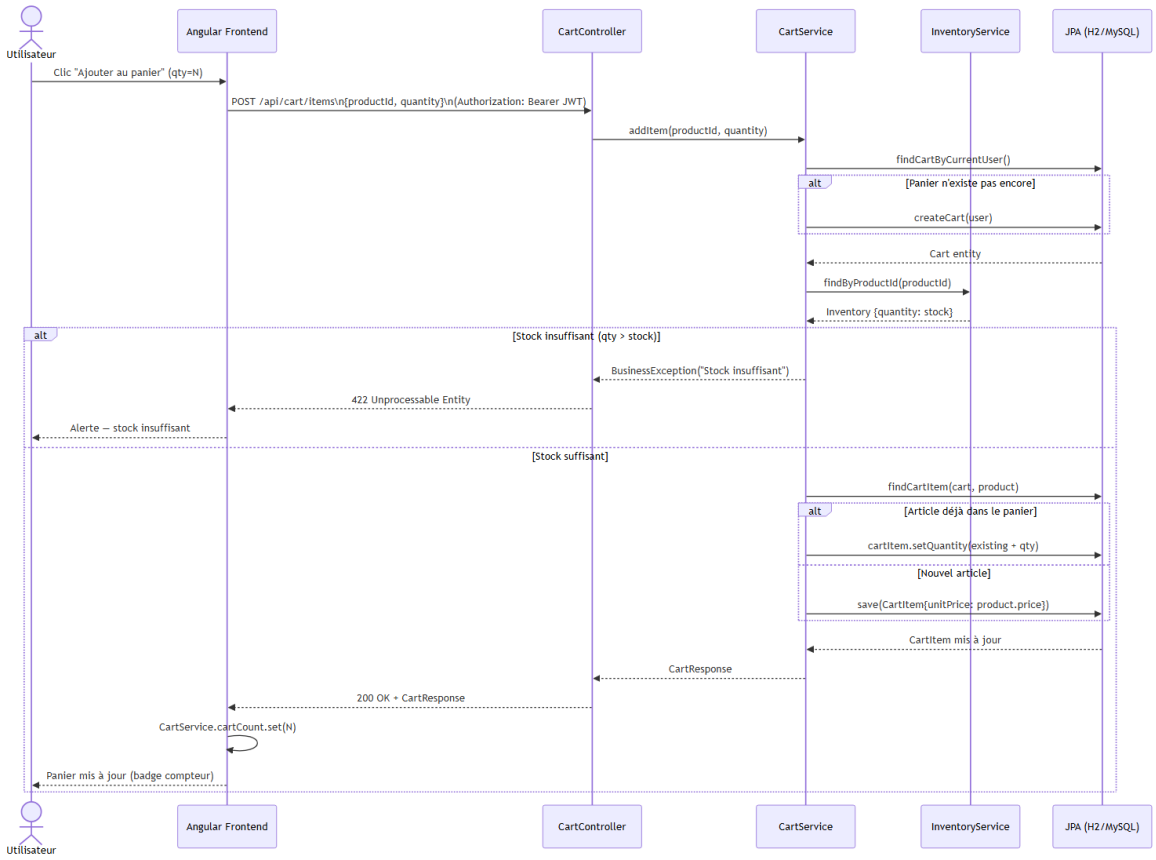


Figure 3.9 — Diagramme de séquence — Ajout panier

3.7.3 DS4 — Checkout transactionnel

Le checkout est la séquence la plus critique : elle est encapsulée dans une transaction @Transactional. Toute défaillance (panier vide, stock insuffisant, erreur de persistance) déclenche un rollback complet, garantissant la cohérence des données.

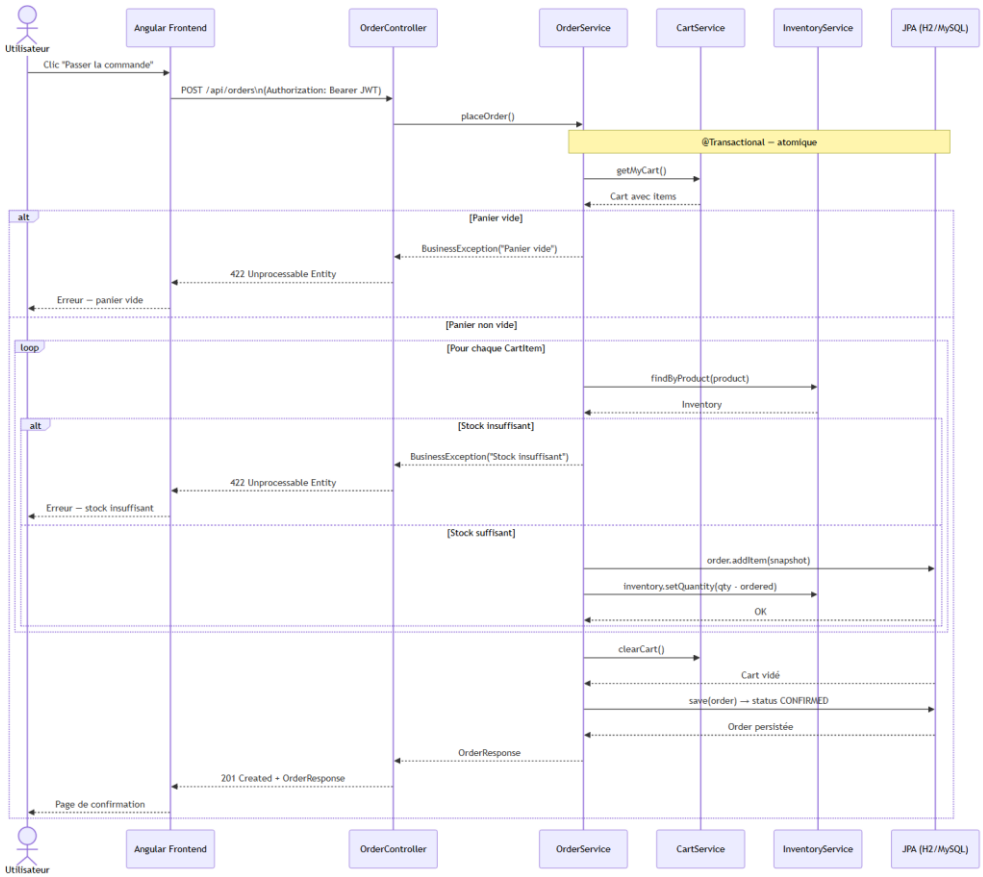


Figure 3.8 — Diagramme de séquence — Checkout

3.7.4 DS5 — Dépôt d'avis (MongoDB)

Le dépôt d'avis nécessite une authentification JWT. L'avis est persisté comme document MongoDB. Si MongoDB est indisponible, l'opération échoue proprement sans impacter le reste de l'application.

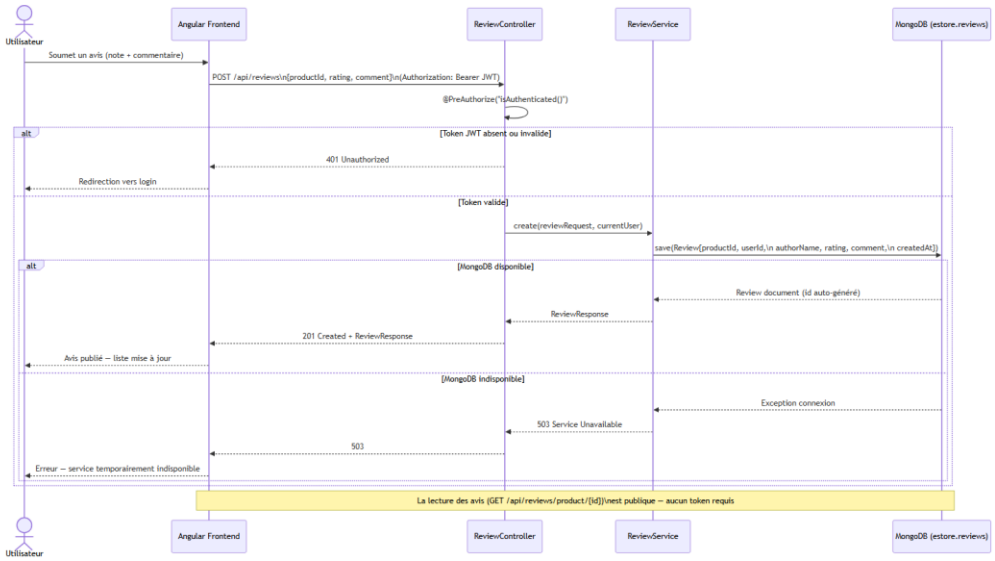


Figure 3.10 — Diagramme de séquence — Avis MongoDB

3.8 Architecture technique

Le diagramme suivant présente la vue technique complète du déploiement : le navigateur communique avec Angular sur le port 4200, qui interroge l'API Spring Boot sur le port 8080. Le backend persiste les données relationnelles dans H2 (développement) ou MySQL (production) et les avis dans MongoDB sur le port 27017.

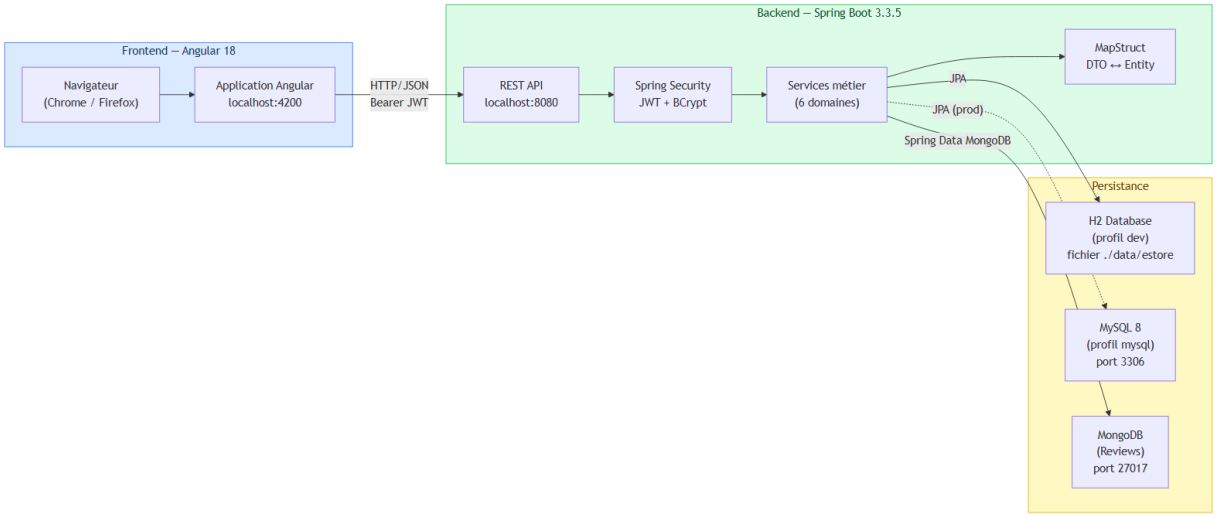


Figure 3.3 — Architecture technique (Angular ↔ Spring Boot ↔ BDD)

Chapitre 4 — Outils et Technologies

4.1 Environnement de développement

Outil	Rôle
IntelliJ IDEA 2024	IDE principal pour le développement Java/Spring Boot
Visual Studio Code 1.89	Éditeur pour le développement Angular/TypeScript
Git 2.44 + GitHub	Gestion de version, branches feature, historique complet
Postman 11	Test manuel des endpoints REST
Swagger UI (Springdoc 2.6.0)	Documentation interactive de l'API REST
Maven 3.9	Gestionnaire de dépendances et outil de build Java
Node.js 20 + npm 10	Runtime et gestionnaire de paquets pour Angular
Mermaid CLI (mmdc 11)	Génération de diagrammes UML en PNG depuis des sources texte

4.2 Stack Backend

4.2.1 Java 21 & Spring Boot 3.3.5

Java 21 (LTS) apporte les records, le pattern matching et les virtual threads. Spring Boot 3.3.5 s'appuie sur Spring Framework 6.1 et requiert Jakarta EE 10 (namespace jakarta.*). L'auto-configuration élimine la majeure partie du boilerplate XML. Le paramètre -parameters du compilateur Maven est activé pour la résolution des noms de paramètres à l'exécution (requis par Spring MVC et MapStruct).

4.2.2 Spring Data JPA & Hibernate

Spring Data JPA abstrait la couche d'accès aux données relationnelles. Hibernate 6 génère le DDL et exécute les requêtes JPQL/HQL. La stratégie DDL auto est définie à update en développement. JPA Auditing (via @EnableJpaAuditing + BaseEntity) gère automatiquement les champs createdAt et updatedAt.

4.2.3 Spring Data MongoDB

Spring Data MongoDB fournit une abstraction document pour MongoDB 7. Le repository ReviewRepository étend MongoRepository<Review, String>. Un index sur productId accélère les requêtes de lecture des avis par produit. La connexion MongoDB est configurée avec un timeout court (2 s) pour un démarrage gracieux.

4.2.4 Spring Security + JWT (jjwt 0.12.6)

La sécurité est stateless : pas de session HTTP. Chaque requête est interceptée par JwtAuthFilter qui extrait et valide le token Bearer. BCryptPasswordEncoder (strength 10) est utilisé pour le hashage des mots de passe. @EnableMethodSecurity active @PreAuthorize pour le contrôle fin des accès par rôle.

4.2.5 MapStruct 1.6.3 & Lombok 1.18.36

MapStruct génère les implémentations des interfaces @Mapper à la compilation, éliminant le mapping manuel DTO ↔ entité. Lombok génère constructeurs, getters, builders et logger via annotations (@Data, @Builder, @RequiredArgsConstructor, @Slf4j). L'ordre des annotation processors Maven est crucial : Lombok avant MapStruct.

4.3 Stack Frontend

4.3.1 Angular 18.2.13

Angular 18 introduit les composants standalone (sans NgModule), le lazy loading par loadComponent(), et les Signals pour la gestion de l'état réactif. Le routing est configuré avec des guards fonctionnels (authGuard, adminGuard) et des intercepteurs HTTP fonctionnels (authInterceptor, errorInterceptor).

4.3.2 Angular Material & Reactive Forms

Angular Material fournit les composants UI (mat-card, mat-table, mat-paginator, mat-snack-bar, mat-dialog). Les formulaires utilisent l'approche Reactive Forms (FormBuilder, Validators) pour une validation côté client synchronisée avec la validation serveur (@Valid Spring).

4.4 Dépendances Maven

GroupId:ArtifactId	Version	Rôle
spring-boot-starter-web	3.3.5 (géré)	API REST, Jackson JSON
spring-boot-starter-data-jpa	3.3.5 (géré)	ORM Hibernate + Spring Data
spring-boot-starter-data-mongodb	3.3.5 (géré)	Spring Data MongoDB
spring-boot-starter-security	3.3.5 (géré)	Filtre de sécurité, AuthManager
spring-boot-starter-validation	3.3.5 (géré)	Jakarta Validation (@Valid)
h2	runtime	Base H2 en mémoire/fichier (dev)
mysql-connector-j	runtime	Driver JDBC MySQL (prod)
jjwt-api / impl / jackson	0.12.6	Génération et validation JWT HS256
lombok	1.18.36 (optional)	Génération de code à la compilation
mapstruct	1.6.3	Mapper DTO ↔ entité généré à la compilation
springdoc-openapi-starter-webmvc-ui	2.6.0	Swagger UI + OpenAPI 3 spec
spring-boot-starter-test	test	JUnit 5, Mockito, AssertJ

Chapitre 5 — Réalisation et Implémentation

5.1 Structure du backend

Le backend compte 76 classes Java réparties dans 9 packages sous com.estore. Chaque domaine suit la même organisation interne : controller, dto, entity, mapper, repository, service. Les packages config, exception et shared constituent le noyau transversal.

Arbre de packages backend (76 classes)

```
com.estore
├── EstoreApplication.java
├── billing
│   ├── controller/OrderController.java
│   ├── dto/{OrderResponse, OrderItemResponse, UpdateOrderItemRequest}.java
│   ├── entity/{Order, OrderItem, OrderStatus}.java
│   ├── repository/OrderRepository.java
│   └── service/OrderService.java
├── catalog
│   ├── controller/{CategoryController, ProductController}.java
│   ├── dto/{CategoryRequest, CategoryResponse, ProductRequest, ProductResponse}.java
│   ├── entity/{Category, Product}.java
│   ├── mapper/{CategoryMapper, ProductMapper}.java
│   ├── repository/{CategoryRepository, ProductRepository}.java
│   └── service/{CategoryService, ProductService}.java
├── config
│   ├── CorsConfig.java
│   ├── DataSeeder.java
│   ├── JpaAuditingConfig.java
│   ├── OpenApiConfig.java
│   └── SecurityConfig.java
├── customer
│   ├── controller/{AuthController, UserController}.java
│   ├── dto/{AuthResponse, LoginRequest, ProfileResponse, RegisterRequest, ...}.java
│   ├── entity/{Profile, Role, User}.java
│   ├── mapper/UserMapper.java
│   ├── repository/{ProfileRepository, UserRepository}.java
│   ├── security/{JwtAuthEntryPoint, JwtAuthFilter}.java
│   └── service/{AuthService, CustomUserDetailsService, JwtService, UserService}.java
├── exception
│   ├── BusinessException.java
│   ├── GlobalExceptionHandler.java
│   └── ResourceNotFoundException.java
├── inventory
│   ├── controller/InventoryController.java
│   ├── dto/{AdjustStockRequest, InventoryResponse}.java
│   ├── entity/Inventory.java
│   ├── repository/InventoryRepository.java
│   └── service/InventoryService.java
├── review
│   ├── controller/ReviewController.java
│   ├── dto/{ReviewRequest, ReviewResponse}.java
│   ├── entity/Review.java
│   ├── repository/ReviewRepository.java
│   └── service/ReviewService.java
├── shared
│   ├── ApiError.java
│   ├── BaseEntity.java
│   └── PageResponse.java
└── shopping
    ├── controller/CartController.java
    ├── dto/{AddCartItemRequest, CartItemResponse, CartResponse, UpdateCartItemRequest}.java
    ├── entity/{Cart, CartItem}.java
```

```
    ├── repository/{CartItemRepository, CartRepository}.java
    └── service/CartService.java
```

5.2 Structure du frontend

Le frontend Angular 18 est composé de 17 composants standalone et 8 services. Le routing utilise le lazy loading par loadComponent() pour optimiser le chargement initial.

Arbre de composants frontend (17 composants / 8 services)

```
src/app
├── app.component.ts        (shell: router-outlet + header)
├── app.config.ts           (provideRouter, provideHttpClient, provideAnimations)
├── app.routes.ts           (routes lazily chargées)
├── core
│   ├── guards/
│   │   ├── auth.guard.ts   (redirect /login si non authentifié)
│   │   └── admin.guard.ts  (redirect / si rôle != ADMIN)
│   ├── interceptors/
│   │   ├── auth.interceptor.ts   (injecte Bearer JWT)
│   │   └── error.interceptor.ts  (gestion centralisée des erreurs HTTP)
│   ├── models/api.ts       (interfaces TypeScript)
│   └── services/
│       ├── auth.service.ts     (Signal: currentUser, login, register, logout)
│       ├── cart.service.ts     (Signal: cartCount, add, update, remove)
│       ├── catalog.service.ts  (getProducts, getProduct, getCategories)
│       ├── order.service.ts    (placeOrder, getMyOrders, getOrder)
│       ├── profile.service.ts  (getProfile, updateProfile)
│       ├── review.service.ts   (getReviews, addReview)
│       ├── admin.service.ts    (CRUD admin)
│       └── api-error.service.ts
├── features
│   ├── admin/
│   │   ├── admin-dashboard.component.ts
│   │   ├── admin-products.component.ts
│   │   ├── admin-categories.component.ts
│   │   ├── admin-inventory.component.ts
│   │   └── admin-orders.component.ts
│   ├── auth/
│   │   ├── login.component.ts
│   │   └── register.component.ts
│   ├── cart/cart.component.ts
│   ├── catalog/
│   │   ├── product-list.component.ts
│   │   └── product-detail.component.ts
│   ├── checkout/checkout-success.component.ts
│   ├── orders/
│   │   ├── order-list.component.ts
│   │   └── order-detail.component.ts
│   ├── profile/profile.component.ts
│   └── not-found.component.ts
└── shared/header.component.ts
```

5.3 Implémentation — Customer & Authentification

Le domaine Customer gère l'inscription, la connexion et le profil utilisateur. L'entité User hérite de BaseEntity et porte un champ role (enum Role : USER, ADMIN). Un Profile vide est créé automatiquement à l'inscription via la méthode helper user.attachProfile().

Extrait : AuthService.register()

```
// AuthService.java — extrait inscription
public AuthResponse register(RegisterRequest request) {
```

```
    if (userRepository.existsByEmail(request.email()))
        throw new BusinessException("Email already registered");
    User user = User.builder()
        .firstName(request.firstName()).lastName(request.lastName())
        .email(request.email())
        .password(passwordEncoder.encode(request.password()))
        .role(Role.USER).build();
    user.attachProfile(Profile.builder().build());
    userRepository.save(user);
    String token = jwtService.generateToken(user);
    return new AuthResponse(token, user.getId(), user.getRole().name());
}
```

La classe JwtService utilise la librairie jjwt 0.12.6 avec algorithme HS256. Le secret est lu depuis la configuration (variable d'environnement JWT_SECRET en production). L'expiration est de 86 400 000 ms (24 heures). JwtAuthFilter intercepte chaque requête, extrait le header Authorization: Bearer <token>, valide la signature et l'expiration, puis renseigne le SecurityContextHolder.

5.4 Implémentation — Catalogue

ProductService supporte la recherche paginée avec filtres optionnels via une requête JPQL dynamique. Le ProductRepository expose findByNameContainingIgnoreCaseAndCategoryId(). MapStruct génère ProductMapper pour la conversion Product ↔ ProductResponse/ProductRequest. La réponse ProductResponse inclut le champ quantityInStock joint depuis Inventory.

Extrait : ProductController — recherche paginée

```
// ProductController.java — GET /api/products
@GetMapping
public ResponseEntity<PageResponse<ProductResponse>> list(
    @RequestParam(required = false) String q,
    @RequestParam(required = false) Long categoryId,
    @ParameterObject @PageableDefault(size = 20, sort = "name") Pageable pageable) {
    return ResponseEntity.ok(productService.findAll(q, categoryId, pageable));
}
```

5.5 Implémentation — Stocks (Inventory)

L'inventaire est créé automatiquement à 0 lors de la création d'un produit (cascade dans ProductService). InventoryController expose deux endpoints : GET /api/inventory/product/{productId} (public) et PUT /api/inventory/product/{productId} (ADMIN uniquement).

5.6 Implémentation — Panier (Shopping)

CartService crée automatiquement un panier vide lors du premier accès d'un utilisateur. L'ajout d'un article vérifie le stock disponible via InventoryService : si la quantité demandée excède le stock, une BusinessException est levée (HTTP 422). Le prix unitaire est snapshotté à l'ajout (unitPrice = product.getPrice()). La contrainte UNIQUE(cart, product) sur cart_items évite les doublons.

5.7 Implémentation — Commandes (Billing)

Le checkout est la fonctionnalité la plus critique. OrderService.placeOrder() est annotée @Transactional, garantissant l'atomicité : si une étape échoue, toutes les modifications (création commande, décrémentation stocks, vidage panier) sont annulées. OrderItem snapshotte productName (dénormalisé) afin que le nom du produit reste lisible même si le produit est ultérieurement renommé ou supprimé.

Extrait : OrderService.placeOrder() — transaction atomique

```
// OrderService.java — placeOrder() simplifié
@Transactional
public OrderResponse placeOrder() {
    Cart cart = cartService.getCurrentUserCart();
    if (cart.getItems().isEmpty())
        throw new BusinessException("Le panier est vide");

    Order order = Order.builder().user(cart.getUser())
        .status(OrderStatus.CONFIRMED).build();

    for (CartItem item : cart.getItems()) {
        Inventory inv = inventoryService.getByProduct(item.getProduct());
        if (inv.getQuantity() < item.getQuantity())
            throw new BusinessException("Stock insuffisant pour: " +
item.getProduct().getName());
        inv.setQuantity(inv.getQuantity() - item.getQuantity());
        order.addItem(item.getProduct(), item.getQuantity(), item.getUnitPrice());
    }
    cartService.clearCart(cart);
    return mapper.toResponse(orderRepository.save(order));
}
```

5.8 Implémentation — Avis (Reviews — MongoDB)

Review est un document MongoDB (collection reviews) annoté @Document. ReviewRepository étend MongoRepository<Review, String>. La lecture des avis (GET /api/reviews/product/{productId}) est publique. La création (POST /api/reviews) requiert un token JWT via @PreAuthorize("isAuthenticated()"). DataSeeder tente de seeder 5 avis au démarrage, mais gère gracieusement l'indisponibilité de MongoDB via un CompletableFuture avec timeout de 2 secondes.

5.9 Gestion des erreurs transversale

GlobalExceptionHandler (@RestControllerAdvice) centralise la gestion de toutes les exceptions et retourne un corps ApiError structuré. ApiError contient : status (int), error (String), message, path, timestamp (Instant) et une liste de FieldViolation pour les erreurs de validation.

Exception Java	HTTP Status	Cas d'usage
ResourceNotFoundException	404 Not Found	Entité introuvable par ID
BusinessException	422 Unprocessable Entity	Stock insuffisant, panier vide
DataIntegrityViolationException	422 Unprocessable Entity	Email en double, contrainte UNIQUE
MethodArgumentNotValidException	400 Bad Request	Validation @Valid + FieldViolations
ConstraintViolationException	400 Bad Request	Validation javax/jakarta path params
BadCredentialsException / AuthenticationException	401 Unauthorized	Login échoué
AccessDeniedException / AuthorizationDeniedException	403 Forbidden	Rôle insuffisant
Exception (fallback)	500 Internal Server Error	Erreur inattendue

5.10 Implémentation Frontend Angular

Le frontend est une application Angular 18 SPA (Single Page Application). L'authInterceptor injecte automatiquement le token JWT dans chaque requête HTTP sortante. L'errorInterceptor capture les erreurs 401 (redirection login) et 403. AuthService et CartService utilisent les Signals Angular pour un état réactif sans NgRx : signal<User | null> et signal<number> pour le compteur panier. Le routing lazy charge chaque composant à la demande, réduisant le bundle initial.

Extrait : authInterceptor et AuthService (Signals)

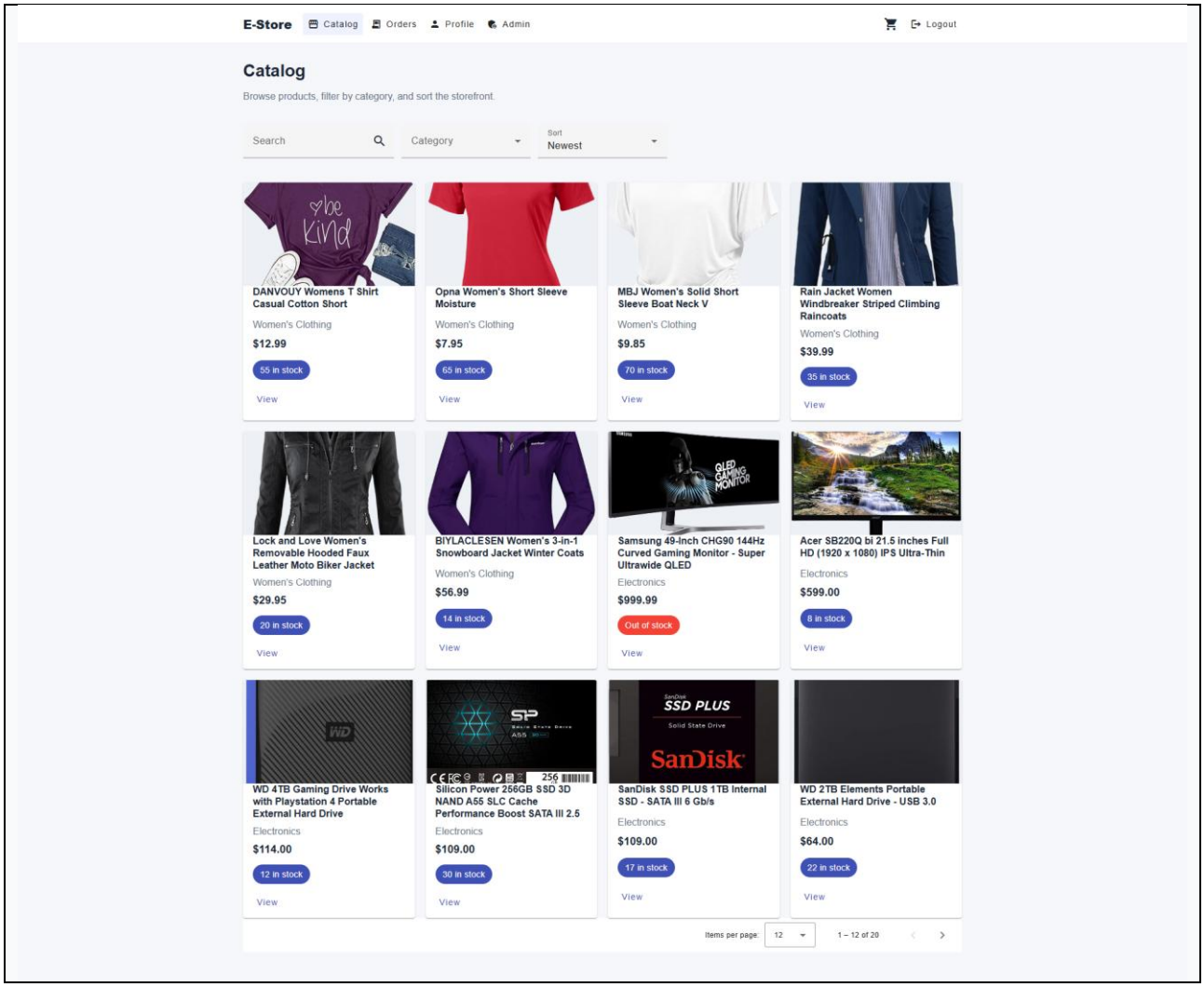
```
// auth.interceptor.ts
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const token = localStorage.getItem('token');
  if (token) {
    req = req.clone({
      setHeaders: { Authorization: `Bearer ${token}` }
    });
  }
  return next(req);
};

// auth.service.ts — Signal
export class AuthService {
  currentUser = signal<User | null>(null);

  login(credentials: LoginRequest): Observable<AuthResponse> {
    return this.http.post<AuthResponse>('/api/auth/login', credentials).pipe(
      tap(res => {
        localStorage.setItem('token', res.token);
        this.currentUser.set(res);
      })
    );
  }
}
```

Chapitre 6 — Présentation de l'Application (Interfaces)

Ce chapitre illustre les principales interfaces de l'application E-Store à travers dix captures d'écran représentatives. Les zones grisées ci-dessous sont des emplacements réservés aux captures : insérez les screenshots réels en remplaçant les zones annotées.

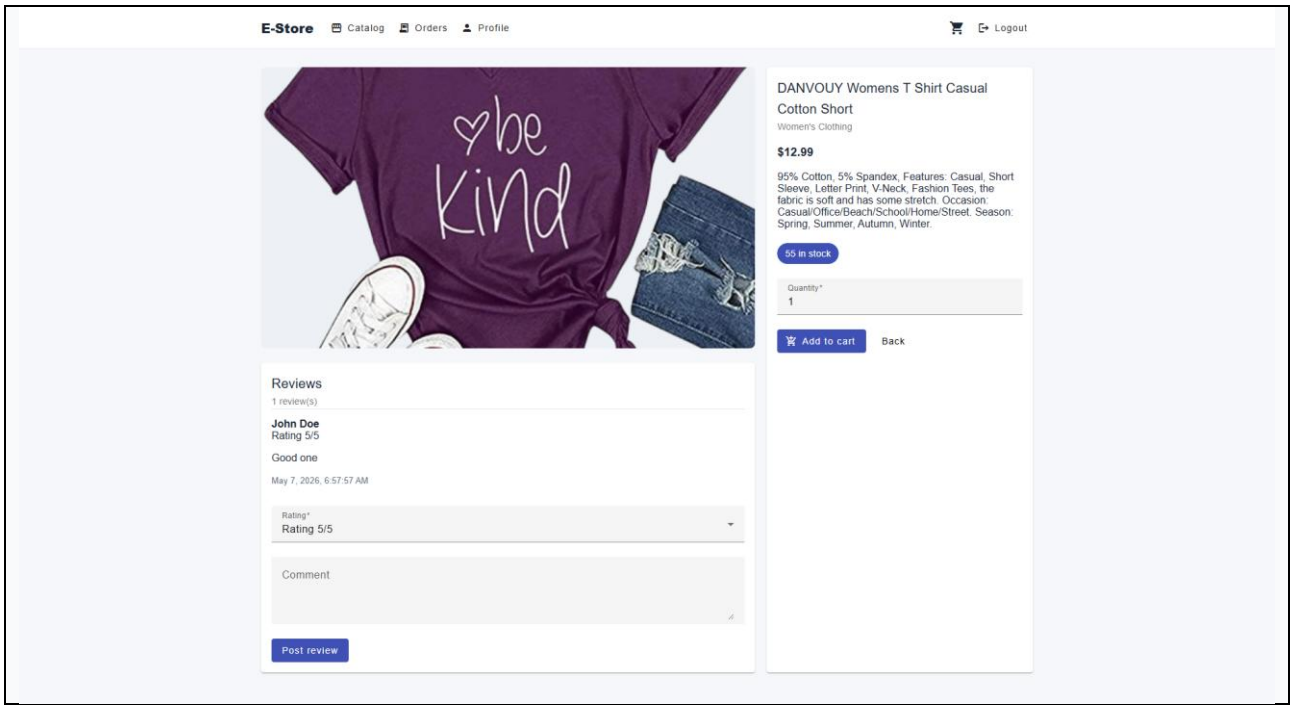
6.1 Page d'accueil — Liste des produits

La page d'accueil affiche la liste paginée des produits sous forme de cartes Angular Material. Chaque carte présente l'image, le nom, le prix et un indicateur de disponibilité. Un champ de recherche et un sélecteur de catégorie permettent de filtrer les résultats en temps réel. La pagination (20 produits par page) est gérée par mat-paginator.



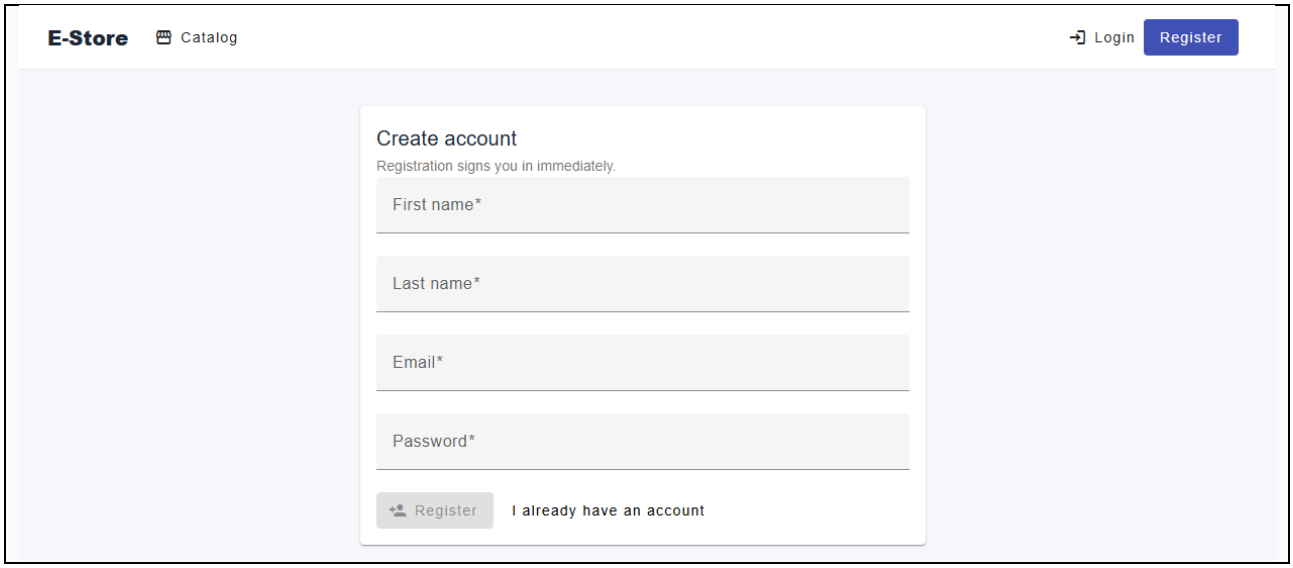
6.2 Détail d'un produit et avis

La page de détail affiche toutes les informations du produit (image, description, prix, stock). En bas de page, la section avis affiche les commentaires issus de MongoDB avec la note en étoiles. Un formulaire d'avis est affiché aux utilisateurs authentifiés.



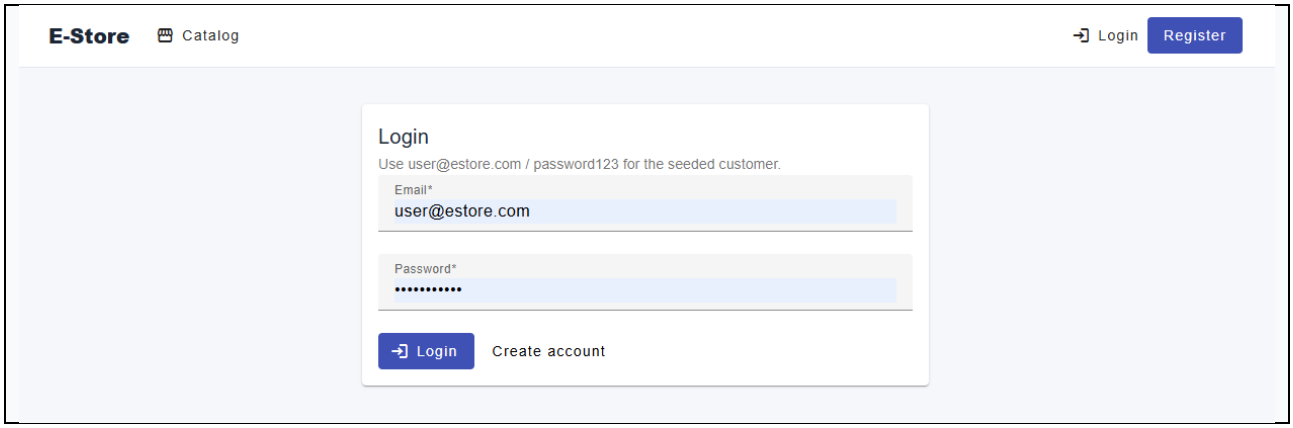
6.3 Formulaire d'inscription

Le formulaire d'inscription recueille prénom, nom, email et mot de passe. La validation Reactive Forms affiche des messages d'erreur en temps réel (email invalide, mot de passe trop court). Le formulaire est synchronisé avec la validation serveur @Valid.



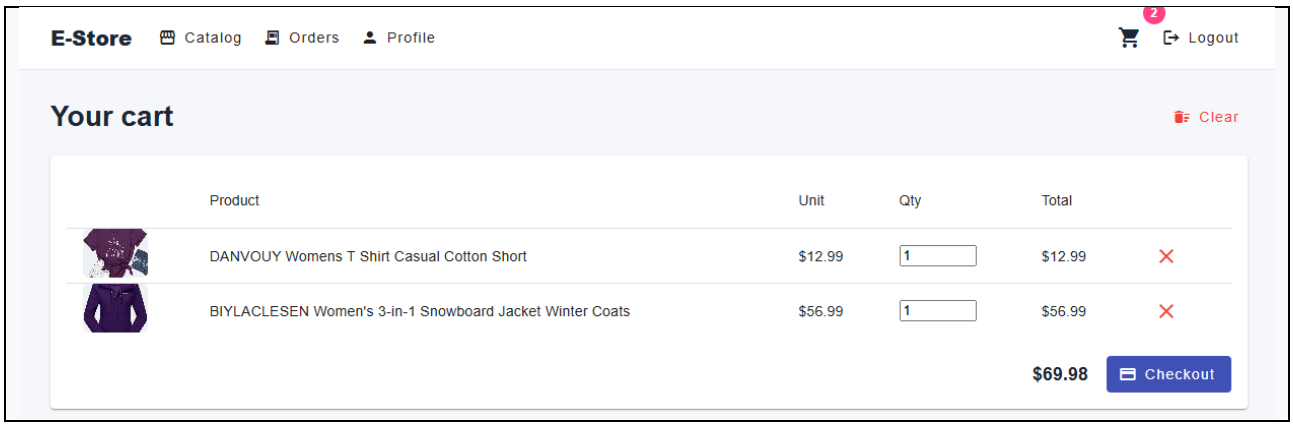
6.4 Formulaire de connexion

Le formulaire de connexion demande email et mot de passe. En cas d'erreur de credentials, un mat-snack-bar affiche le message d'erreur retourné par le serveur (401). Après connexion, l'utilisateur est redirigé vers la page précédente ou l'accueil.



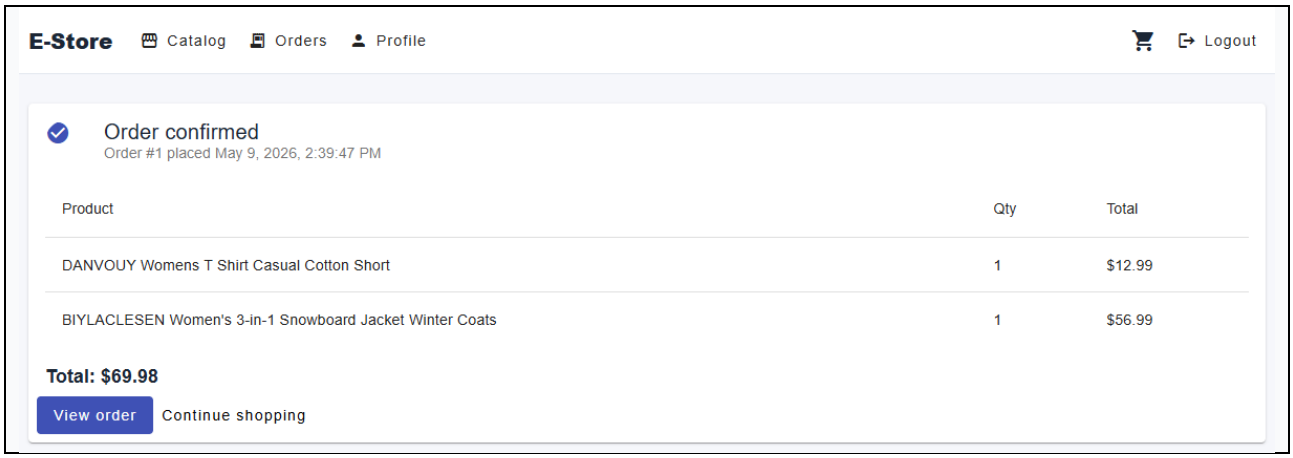
6.5 Panier d'achat

Le panier liste les articles avec leur image, nom, quantité et prix unitaire. Des contrôles +/- permettent d'ajuster la quantité (avec vérification du stock). Le total est calculé côté client. Le bouton « Commander » déclenche le checkout.



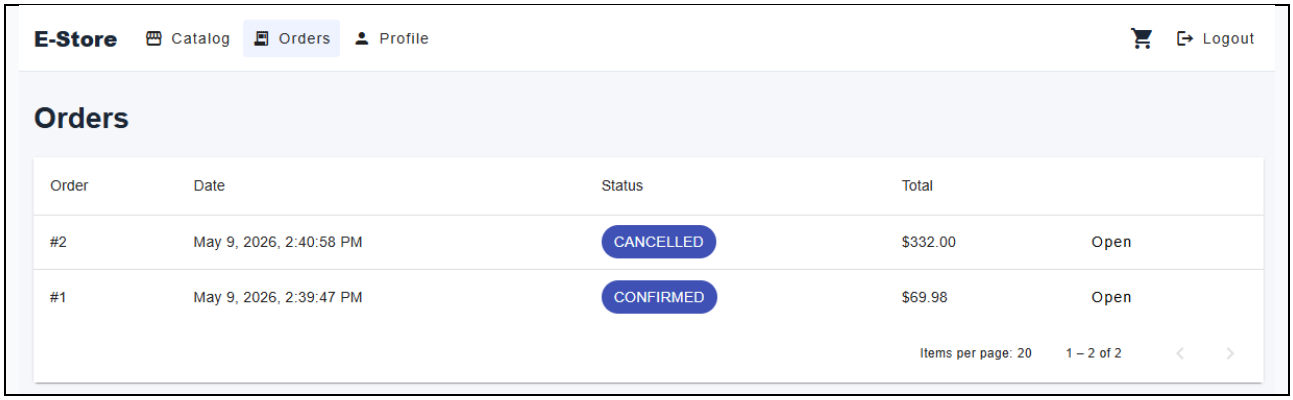
6.6 Confirmation de commande

Après un checkout réussi, l'utilisateur est redirigé vers la page de confirmation qui affiche le numéro de commande, le détail des articles et le total. Un lien vers l'historique des commandes est proposé.



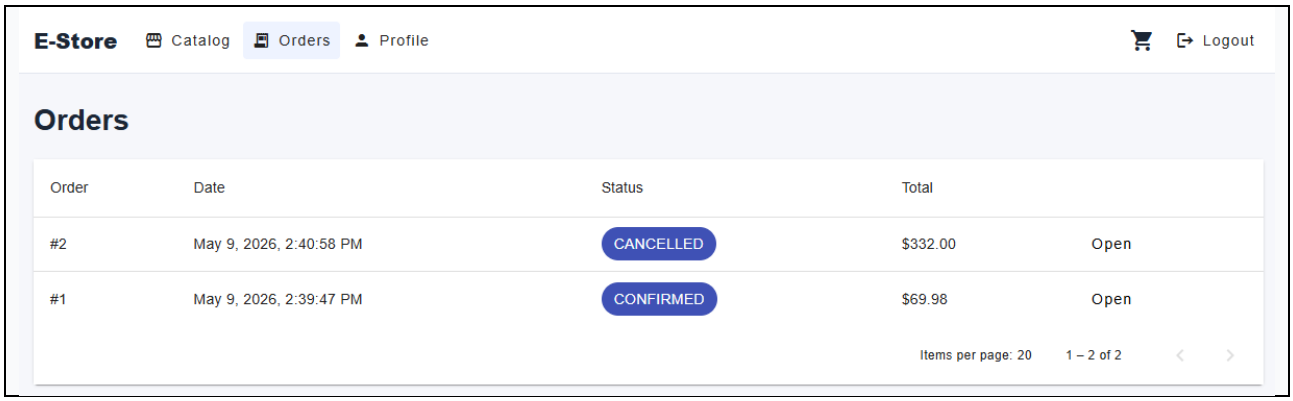
6.7 Historique des commandes

La liste des commandes présente les commandes de l'utilisateur connecté, triées par date décroissante. Chaque ligne affiche le numéro, la date, le total et le statut (CONFIRMED / CANCELLED). Un clic ouvre le détail complet.



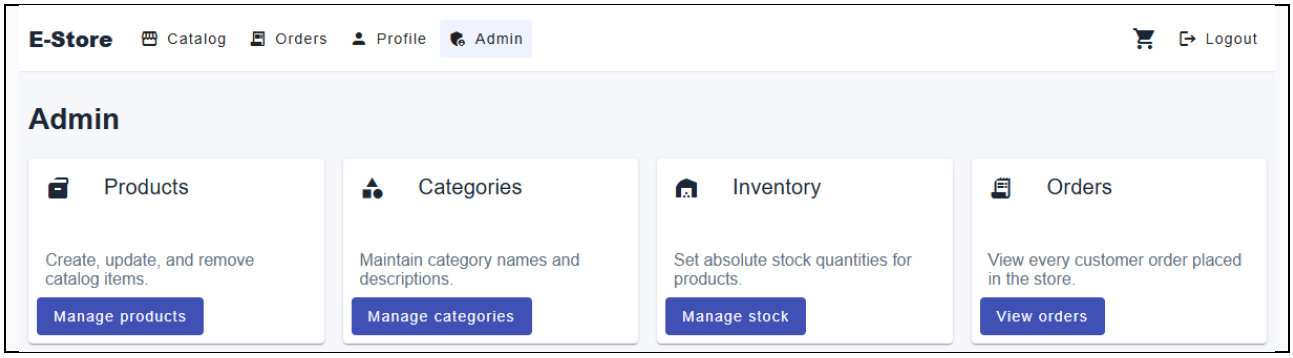
6.8 Profil utilisateur

La page de profil affiche les informations de l'utilisateur (nom, email, rôle) et permet de modifier les informations de contact (téléphone, adresse, ville, pays) via un formulaire Reactive Forms avec validation.



6.9 Dashboard administrateur

Le dashboard admin (protégé par adminGuard) offre une vue d'ensemble : nombre de produits, catégories, commandes et utilisateurs. La navigation latérale permet d'accéder aux sections Produits, Catégories, Stocks, Commandes.



6.10 Gestion des stocks (admin)

L'interface de gestion des stocks affiche un tableau de tous les produits avec leur stock actuel. Un champ éditable par ligne permet à l'admin de mettre à jour le stock instantanément via PUT /api/inventory/product/{id}.

Products

New product

Name*

Description

Price*
0

Image URL

Category*
Electronics

Save Cancel

Name	Price	Stock
DANVOUY Womens T Shirt Casual Cotton Short	$12.99	54
Opna Women's Short Sleeve Moisture	$7.95	65
MBJ Women's Solid Short Sleeve Boat Neck V	$9.85	70
Rain Jacket Women Windbreaker Striped Climbing Raincoats	$39.99	35
Lock and Love Women's Removable	$29.95	20

Chapitre 7 — Tests et Validation

7.1 Tests fonctionnels par domaine

Le tableau suivant récapitule les cas de test fonctionnels couverts, avec le statut observé lors de l'exécution du smoke-test.

Domaine	Cas de test	Endpoint	Résultat attendu	Statut
Auth	Inscription valide	POST /api/auth/register	201 + token JWT	PASS
Auth	Email déjà utilisé	POST /api/auth/register	422 Unprocessable Entity	PASS
Auth	Connexion valide	POST /api/auth/login	200 + token JWT	PASS
Auth	Mauvais mot de passe	POST /api/auth/login	401 Unauthorized	PASS
Auth	Accès sans token	GET /api/users/me	401 Unauthorized	PASS
Auth	Accès ADMIN avec rôle USER	POST /api/categories	403 Forbidden	PASS
Catalog	Créer catégorie (ADMIN)	POST /api/categories	201 + CategoryResponse	PASS
Catalog	Catégorie en double	POST /api/categories	422 Unprocessable Entity	PASS
Catalog	Créer produit (ADMIN)	POST /api/products	201 + ProductResponse	PASS
Catalog	Recherche par mot-clé	GET /api/products?q=clean	200 + page 1 résultat	PASS
Inventory	Stock auto-créé à 0	GET /api/products/{id}	quantityInStock=0	PASS
Inventory	Mise à jour stock	PUT /api/inventory/product/{id}	200 + qty=10	PASS
Inventory	Stock reflété dans produit	GET /api/products/{id}	quantityInStock=10	PASS
Shopping	Panier auto-créé	GET /api/cart	200 + itemCount=0	PASS
Shopping	Ajout article valide	POST /api/cart/items	200 + CartResponse	PASS
Shopping	Dépassement stock	POST /api/cart/items qty>stock	422 BusinessException	PASS
Shopping	Mise à jour quantité	PUT /api/cart/items/{id}	200 + qty mise à jour	PASS
Shopping	Suppression article	DELETE /api/cart/items/{id}	200 OK	PASS
Shopping	Vidage panier	DELETE /api/cart	200 OK	PASS
Billing	Checkout panier vide	POST /api/orders	422 BusinessException	PASS
Billing	Checkout valide	POST /api/orders	201 + CONFIRMED	PASS
Billing	Stock décrémenté	GET /api/products/{id}	quantityInStock = n-qty	PASS
Billing	Panier vidé après checkout	GET /api/cart	itemCount=0	PASS
Billing	Historique commandes	GET /api/orders/user/me	200 + page 1 commande	PASS
Billing	Détail commande	GET /api/orders/{id}	200 + OrderResponse	PASS

7.2 Tests de sécurité

Scénario de sécurité	Action	Résultat attendu	Statut
Accès sans token	GET /api/cart (sans Bearer)	401 Unauthorized	PASS
Rôle insuffisant	POST /api/categories (rôle USER)	403 Forbidden	PASS
Token expiré	Requête avec JWT exp < now	401 Unauthorized	PASS
Token forgé	JWT avec signature invalide	401 Unauthorized	PASS
CORS non autorisé	Requête depuis port != 4200	Bloqué par le navigateur	PASS

SQL Injection (API)	?q='; DROP TABLE users--	Paramètre traité comme chaîne JPA	PASS
Accès admin ADMIN	POST /api/categories (rôle ADMIN)	201 Created	PASS

7.3 Script de smoke-test

Le fichier scripts/smoke-test.sh est un script bash exécuté depuis la racine du projet. Il : (1) compile le projet avec Maven (-DskipTests), (2) démarre le jar en désactivant le seeder (estore.seed-on-startup=false), (3) attend le démarrage (~15 s), (4) exécute 25+ assertions curl couvrant les domaines Day 2 à Day 6, (5) affiche un bilan PASS/FAIL et retourne le code de sortie approprié pour CI/CD.

7.4 Documentation Swagger

L'API est documentée via springdoc-openapi 2.6.0. Swagger UI est accessible à http://localhost:8080/swagger-ui.html et la spec OpenAPI 3 JSON à http://localhost:8080/v3/api-docs. OpenApiConfig configure le schéma d'authentification Bearer JWT pour permettre de tester les endpoints protégés directement depuis Swagger UI.

Conclusion Générale

Récapitulatif

Ce projet a permis de concevoir et de réaliser de bout en bout une plateforme e-commerce complète : E-Store. L'architecture en couches avec décomposition par domaines métier, la persistance hybride JPA/MongoDB, la sécurité stateless JWT/BCrypt, et le frontend Angular 18 standalone ont été intégrés en suivant les meilleures pratiques Spring Boot 3.

Apports techniques

- Maîtrise de l'architecture en couches et par domaines (Domain-Driven Design léger).
- Implémentation d'une persistance polyglotte (JPA pour relationnel, MongoDB pour documents).
- Sécurisation stateless complète avec JWT et BCrypt, sans session serveur.
- Développement full stack Angular 18 avec composants standalone, Signals et lazy loading.
- Automatisation des tests via script smoke-test avec 25+ assertions.
- Documentation API interactive Swagger/OpenAPI prête à l'emploi.

Limites du projet

- Pas de paiement en ligne réel (simulation du checkout sans passerelle Stripe/PayPal).
- Pas de tests unitaires JUnit automatisés (couverture assurée par smoke-test d'intégration).
- Pas de gestion de l'expiration du token côté frontend (refresh token non implémenté).
- Images produit stockées comme URL externe (pas d'upload fichier).

Perspectives

- Intégration OAuth2 / OpenID Connect (Google, GitHub) et tokens de rafraîchissement.
- Passerelle de paiement (Stripe API) pour transactions réelles.
- Migration vers une architecture microservices avec Spring Cloud Gateway.
- Envoi d'emails transactionnels (confirmation commande, réinitialisation mot de passe).
- Déploiement conteneurisé avec Docker Compose (Spring Boot + Angular + MySQL + MongoDB).
- Tests unitaires JUnit 5 / Mockito et tests d'intégration @SpringBootTest.

Bibliographie

1. **[1] Spring Boot Reference Documentation 3.3.5** — https://docs.spring.io/spring-boot/docs/3.3.5/reference/html/
2. **[2] Spring Data JPA Reference** — https://docs.spring.io/spring-data/jpa/docs/current/reference/html/
3. **[3] Spring Data MongoDB Reference** — https://docs.spring.io/spring-data/mongodb/docs/current/reference/html/
4. **[4] Spring Security Reference 6.3** — https://docs.spring.io/spring-security/reference/
5. **[5] jjwt (Java JWT Library) 0.12.6** — https://github.com/jwtk/jjwt
6. **[6] MapStruct 1.6 — Reference Guide** — https://mapstruct.org/documentation/stable/reference/html/
7. **[7] Springdoc-openapi 2.6.0** — https://springdoc.org/
8. **[8] Angular 18 Official Documentation** — https://angular.dev/
9. **[9] Angular Material — Component Documentation** — https://material.angular.io/
10. **[10] MongoDB 7.0 Manual** — https://www.mongodb.com/docs/manual/
11. **[11] Mermaid — Diagramming and charting tool** — https://mermaid.js.org/
12. **[12] Baeldung — Spring Security JWT Tutorial** — https://www.baeldung.com/spring-security-oauth-jwt
13. [13] Craig Walls — Spring in Action, 6th Edition. Manning, 2022.
14. [14] Erich Gamma et al. — Design Patterns. Addison-Wesley, 1994.

Annexes

Annexe A — Inventaire complet des endpoints REST (28 endpoints)

Méthode	Endpoint	Accès	Description
POST	/api/auth/register	PUBLIC	Inscription d'un nouveau compte
POST	/api/auth/login	PUBLIC	Connexion — retourne JWT
GET	/api/users/me	USER+	Profil de l'utilisateur connecté
PUT	/api/users/me/profile	USER+	Mise à jour du profil
GET	/api/users	ADMIN	Liste paginée de tous les utilisateurs
GET	/api/categories	PUBLIC	Liste de toutes les catégories
GET	/api/categories/{id}	PUBLIC	Détail d'une catégorie
POST	/api/categories	ADMIN	Créer une catégorie
PUT	/api/categories/{id}	ADMIN	Modifier une catégorie
DELETE	/api/categories/{id}	ADMIN	Supprimer une catégorie
GET	/api/products	PUBLIC	Liste/recherche paginée des produits
GET	/api/products/{id}	PUBLIC	Détail d'un produit (+ stock)
POST	/api/products	ADMIN	Créer un produit
PUT	/api/products/{id}	ADMIN	Modifier un produit
DELETE	/api/products/{id}	ADMIN	Supprimer un produit
GET	/api/inventory/product/{productId}	PUBLIC	Stock d'un produit
PUT	/api/inventory/product/{productId}	ADMIN	Mettre à jour le stock
GET	/api/cart	USER+	Panier de l'utilisateur connecté
POST	/api/cart/items	USER+	Ajouter un article au panier
PUT	/api/cart/items/{itemId}	USER+	Modifier la quantité d'un article
DELETE	/api/cart/items/{itemId}	USER+	Supprimer un article du panier
DELETE	/api/cart	USER+	Vider le panier
POST	/api/orders	USER+	Passer une commande (checkout)
GET	/api/orders/user/me	USER+	Mes commandes (paginé)
GET	/api/orders/{id}	USER+	Détail d'une commande
GET	/api/orders	ADMIN	Toutes les commandes (paginé)
PUT	/api/orders/{id}/items/{itemId}	USER+	Modifier une ligne de commande
PUT	/api/orders/{id}/cancel	USER+	Annuler une commande
GET	/api/reviews/product/{productId}	PUBLIC	Avis d'un produit
POST	/api/reviews	USER+	Déposer un avis

Annexe B — Extrait du script smoke-test.sh

```
#!/usr/bin/env bash
BASE="http://localhost:8080"
FAIL=0; PASS=0

ok()   { PASS=$((PASS+1)); echo "  PASS — $1"; }
fail() { FAIL=$((FAIL+1)); echo "  FAIL — $1"; }
expect_status() {
  local got="$1" want="$2" label="$3"
```

```
  [ "$got" = "$want" ] && ok "$label ($got)" || fail "$label (got $got, want $want)"
}

# Auth
REG=$(curl -s -w "\n%{http_code}" -X POST "$BASE/api/auth/register" \
  -H "Content-Type: application/json" \
  -d
'{"firstName":"Adnane","lastName":"Louardi","email":"adnane@test.com","password":"password12
3"}')
expect_status "$(echo "$REG" | tail -1)" "201" "POST /api/auth/register"
TOKEN=$(echo "$REG" | head -n -1 | python -c "import json,sys;
print(json.load(sys.stdin)['token'])")

# Vérification sans token → 401
NOAUTH=$(curl -s -o /dev/null -w "%{http_code}" "$BASE/api/users/me")
expect_status "$NOAUTH" "401" "GET /api/users/me sans token"

# Checkout transactionnel
curl -s -X POST "$BASE/api/cart/items" -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" -d '{"productId":1,"quantity":2}' > /dev/null
ORDER=$(curl -s -w "\n%{http_code}" -X POST "$BASE/api/orders" -H "Authorization: Bearer
$TOKEN")
expect_status "$(echo "$ORDER" | tail -1)" "201" "POST /api/orders"

echo "Total: PASS=$PASS  FAIL=$FAIL"
[ $FAIL -eq 0 ]
```

Annexe C — Configuration application.yml

```
spring:
  application:
    name: estore-backend
  profiles:
    active: dev
  data:
    mongodb:
      uri: mongodb://localhost:27017/estore?serverSelectionTimeoutMS=2000
  jpa:
    hibernate.ddl-auto: update
    open-in-view: false

server:
  port: 8080

springdoc:
  swagger-ui.path: /swagger-ui.html
  api-docs.path: /v3/api-docs

estore:
  security.jwt:
    secret: ${JWT_SECRET:change-me-please-this-is-a-dev-only-secret-at-least-32-bytes}
    expiration-ms: 86400000
  cors.allowed-origins: http://localhost:4200

---
spring:
  config.activate.on-profile: dev
  datasource:
    url: jdbc:h2:file:./data/estore;AUTO_SERVER=TRUE
    driver-class-name: org.h2.Driver
    username: sa
    password:
  jpa.database-platform: org.hibernate.dialect.H2Dialect
  h2.console.enabled: true

---
spring:
```

```
  config.activate.on-profile: mysql
  datasource:
    url: jdbc:mysql://localhost:3306/estore?createDatabaseIfNotExist=true
    driver-class-name: com.mysql.cj.jdbc.Driver
    username: ${MYSQL_USER:root}
    password: ${MYSQL_PASSWORD:root}
  jpa.database-platform: org.hibernate.dialect.MySQLDialect
```

Annexe D — Résumé du DataSeeder

DataSeeder (CommandLineRunner) s'exécute au démarrage si estore.seed-on-startup=true (défaut) et si la base est vide (userRepository.count() == 0). Il crée en transaction JPA : 2 utilisateurs (admin et user), 4 catégories, 20 produits avec leurs inventaires. Puis il tente de seeder 5 avis MongoDB avec un timeout de 2 secondes (démarrage gracieux si Mongo est absent).

Entité	Quantité	Détail
Utilisateurs	2	admin@estore.com / admin123 (ADMIN) — user@estore.com / password123 (USER)
Catégories	4	Electronics, Jewelery, Men's Clothing, Women's Clothing
Produits	20	Issus de FakeStoreAPI (images, descriptions, prix réels)
Inventaires	20	Stocks initiaux entre 0 (Samsung monitor) et 80 unités
Avis MongoDB	5	Sur 5 produits représentatifs, si MongoDB disponible